Separable NM-ROM: architecture, training, and why linear problems need no quadrature

A study deck — what is learned, what is frozen, what is exact, what is sampled

Tunable-NM-ROM project

29 August 2026 — extended the same day with the quadratic-tensor rung

Numbers: single-seed, f64, gate-checked A100 runs; tables reproduced from the generated presentation notes (28 Aug 2026).

The question this deck answers

The setup

A PDE solution $u(x)$ on a fine grid (n up to 2.6×10^5 points) is represented by a **small latent vector** $z \in \mathbb{R}^K$ ($K = 8$ or 16) through a **decoder**.

The reduced-order model (ROM) solves the PDE *for* z, never for the grid values directly.

The catch

The PDE residual is a *sum over the whole grid*. If every solver iteration had to decode the field on the grid and sum it, the ROM would cost as much as the full solver.

What we will see

1. How the decoder is built so the grid can be *factored out*.
2. How it is trained (stage 1), then frozen.
3. **Linear PDE (Poisson):** the whole residual collapses to a small precomputed table — *no sampling, no decoding, exact*.
4. **Nonlinear PDE (Burgers):** one term refuses to collapse — we *sample* it at a few points and *learn where* (stage 2).
5. **The next rung (new, 29 Aug):** a *quadratic* term collapses too — into a precomputed table — so Burgers becomes sample-free, and stage 2 disappears.

The separable decoder

$$u(x;z) \;=\; bc(x)\,\big\langle\, g(x),\; h(z)\,\big\rangle \;=\; bc(x)\sum_{j=1}^{R} g_j(x)\,h_j(z)$$

- $g(x) \in \mathbb{R}^R$ — the **bank**: R spatial *shapes* (Fourier features → small MLP). Depends on x only.
- $h(z) \in \mathbb{R}^R$ — the **head**: MLP + linear skip from the latent to R *mixing amounts*. Depends on z only.
- $bc(x)$ — a fixed mask that makes the boundary condition hold exactly.
- $R = 32$ (Burgers), $R = 64$–96 (Poisson).

Why "separable" matters

The two tracks never see each other's input. Evaluate the bank on any point set once and store it as a table

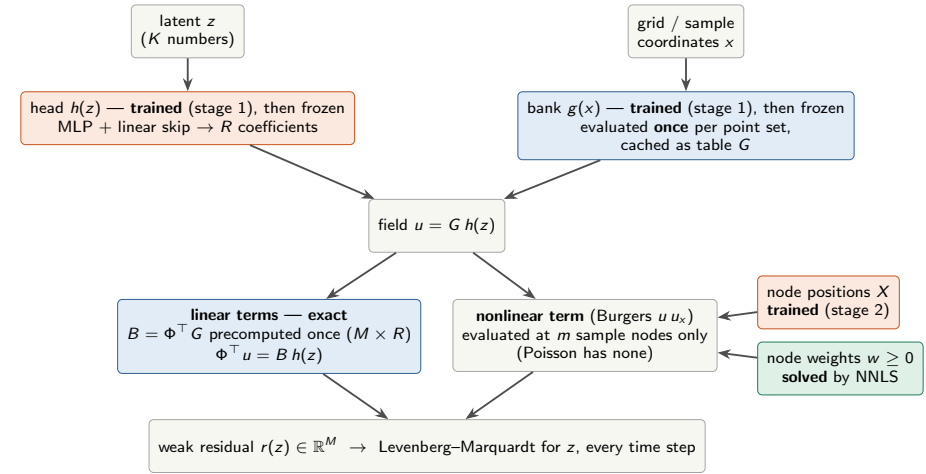
$$G \in \mathbb{R}^{n\times R}, \qquad G_{ij} = bc(x_i)\,g_j(x_i).$$

Then the field on that point set is just

$$u(z) \;=\; G\,h(z) \qquad \text{(table} \times \text{coefficient vector)}.$$

The field is a mix of R fixed shapes; only the R mixing amounts depend on z. Everything downstream exploits exactly this.

Architecture diagram — who is trained, who is frozen, who is solved



Stage 1 — training the decoder (auto-decoder)

$$L_1 = \underbrace{\frac{\text{mean}_s \; \| G\, h(z_s) - u_s \|^2}{\text{mean} \; \|u\|^2}}_{\text{relative reconstruction}} + \lambda_{\text{orth}} \underbrace{\left\| \frac{G^\top G}{n\, s^2} - I \right\|_F^2}_{\text{keep the bank well-conditioned}}$$

Jointly optimise the bank g, the head h, and **one latent code z_s per training snapshot u_s** (there is no encoder — the codes are free variables). $\lambda_{\text{orth}} = 10^{-4}$; Adam, full batch, warm-up then cosine decay ($10^{-3} \to 10^{-5}$), 40 000 steps.

In words

- First term: "every training snapshot must be reconstructible from *some* 8-number code."
- Second term: "the R shapes should be roughly orthogonal" — a conditioning nudge so the mixing amounts stay well-scaled; it does not change what the decoder can represent.

After stage 1

Bank, head, and codes are **frozen**. Nothing downstream ever changes the decoder. The training codes are discarded; the solver finds its own z at run time.

How the decoder is used online — it defines a manifold, it does not predict

Weak form. Pick M test functions $\Phi \in \mathbb{R}^{n \times M}$ (sine modes = exact eigenvectors of the discrete Laplacian, $\Delta\Phi = \Phi\Lambda$). The residual handed to the solver is the PDE residual projected onto them:

$$r(z) \;=\; \Phi^{\top}\,\mathcal{R}\big(G\,h(z)\big) \;\in \mathbb{R}^{M}, \qquad M = 32 \text{ or } 64.$$

Solve. Each time step: damped Levenberg–Marquardt over the K latent unknowns, driving $r(z) \to 0$; needs r and $\partial r/\partial z$.

The cost question

$\Phi^{\top}\mathcal{R}(u)$ is a **sum over all n grid points**. Done naively, each of thousands of solver iterations would

1. decode $u = G\,h(z)$ on the grid ($n \times R$ flops),
2. apply the PDE operator on the grid,
3. sum against M test modes.

That is the full-order cost, every iteration.

Two ways out

Exact: regroup the arithmetic so the grid disappears (works when the term is linear).
Approximate: estimate the sum from $m \ll n$ sample points with weights — *quadrature* (needed when the term is nonlinear).

Why a linear term needs no sampling

The one fact

A weighted sum of a mix equals the mix of the weighted sums.

$$\Phi^\top u \;=\; \Phi^\top\Big(\sum_j h_j\, G_{:,j}\Big) \;=\; \sum_j h_j\,\big(\Phi^\top G_{:,j}\big) \;=\; \underbrace{(\Phi^\top G)}_{B\,:\,M\times R}\, h(z)$$

The field is a mix of R fixed shapes, $u = \sum_j h_j(z)\, G_{:,j}$, and a linear residual term is a fixed weighted sum of the field — hence the line above. B does not depend on $z \Rightarrow$ compute it **once**, offline, on the full grid. Online: an $M \times R$ matrix times R numbers.

Exact, not approximate. Nothing was estimated; it is the same arithmetic regrouped: $\Phi^\top(G\,h) = (\Phi^\top G)\,h$.

Shopping analogy

Every cart is a mix of the same 32 products in different amounts. You do not re-price the cart item by item each time; you look up 32 unit prices once, and the total is amounts $\times$ unit prices. B is the unit-price sheet: for each of M test modes and each of R shapes, "what this shape contributes to this mode." $64 \times 32 = 2\,048$ numbers, computed once.

Poisson: the whole residual is a small table

$$\boxed{r(z) \;=\; W \odot \big[\, \Lambda\, B\, h(z) \;-\; f_M \,\big]} \qquad \frac{\partial r}{\partial z} \;=\; W \odot \Lambda\, B\, \frac{\partial h}{\partial z}, \qquad B = \Phi^\top G, \quad f_M = \Phi^\top f$$

Poisson: $-\Delta u = f$. Every term is linear in u.

- **Laplacian moved onto the test modes.** Because $\Delta\Phi = \Phi\Lambda$, $\Phi^\top(\Delta u) = (\Delta\Phi)^\top u = \Lambda\,\Phi^\top u$. The operator becomes M known numbers Λ — it never touches the field.
- **Source term** projected once: $f_M = \Phi^\top f$.
- $\partial h/\partial z$: autodiff of an $8 \to 32$ MLP.
- W: fixed per-mode scaling that balances the rows.

What runs online

- the tiny head $h(z)$,
- a 64×32 matvec,
- the head's 32×8 Jacobian.

No grid. No decode. No sample points. No fitted weights. The field $G\,h(z)$ is formed once, at the very end, to write out the answer.

What "0 sample points" means

The sampled (NNLS) path estimates the same M numbers from 256 chosen points — and gets them 5–8 % wrong at solutions. The quadrature-free path computes them exactly and does not need the points at all.

Poisson 2D results ($N = 128/256/512$, same solver, same frozen decoders)

N	path	sample pts	solve ms	solution err	resid. err vs full	grad. cos (min)	setup s
128	full grid	15 876	3.98	3.06e-2	0 (ref)	1.000	—
128	sampled (NNLS)	256	3.14	3.06e-2	7.9e-2	−0.61	28.6
128	**quadrature-free**	**0**	**2.93**	3.06e-2	**4.2e-14**	**1.000**	**0.9**
256	full grid	64 516	4.96	3.14e-2	0	1.000	—
256	sampled (NNLS)	256	2.75	3.14e-2	6.2e-2	−0.60	37.3
256	**quadrature-free**	**0**	**2.68**	3.14e-2	**3.6e-14**	**1.000**	**0.7**
512	full grid	260 100	11.11	3.15e-2	0	1.000	—
512	sampled (NNLS)	256	3.09	3.15e-2	5.5e-2	−0.65	25.4
512	**quadrature-free**	**0**	**2.87**	3.15e-2	**3.3e-14**	**1.000**	**0.8**

Exact: residual matches the full grid to ∼13 digits at every solver solution (zero gate failures).

Fastest and flat in N: full grid grows 4→11 ms; quadrature-free stays ∼2.9 ms.

Nothing to fit: the 25–37 s NNLS quadrature fit disappears; the sampled rule's gradient could even point the wrong way ($\cos < 0$).

Why a nonlinear term breaks the trick

Burgers: $u_t + u\,u_x = \nu\,u_{xx}$. The advection term multiplies the field by itself:

$$\Phi^\top\big(u \odot u_x\big) \;=\; \Phi^\top\Big[\Big(\sum_j h_j G_{:,j}\Big) \odot \Big(\sum_k h_k G'_{:,k}\Big)\Big] \;=\; \sum_{j,k} h_j\, h_k\; \Phi^\top\big(G_{:,j} \odot G'_{:,k}\big)$$

- Mixing does not pass through a product: every *pair* of shapes appears (R^2 terms, $M \times R \times R$ table).
- Worse: our discretisation is **sign-upwind** — the stencil for u_x switches with the sign of u at each point. That is not a polynomial in h; no table can hold it.

Consequence

The only way to know the term's value is to **actually look at the field somewhere**. So we must decode at *some* points — and then we have to choose *which* points and *what weights*: that is quadrature, and its error enters the ROM's error.

But only that one term

Time derivative and diffusion are linear $\Rightarrow$ they still go through $B = \Phi^\top G$, exactly ("exlin"). Sampling touches the advection term and nothing else.

What we do for Burgers: exact linear terms + sampled advection

Backward-Euler step from z^n to z; $N(u) = u\,u_x$ (sign-upwind); m nodes X with weights $w \geq 0$; $\Phi_X, G_X =$ test modes and bank evaluated at the nodes:

$$r(z) \;=\; W \odot \Big[\underbrace{B\big(h(z) - h(z^n)\big)}_{\text{time derivative, exact}} + \Delta t \Big(\underbrace{\Phi_X^\top \big(w \odot N_X(G_X h(z))\big)}_{\text{advection, sampled at } m \text{ nodes}} + \underbrace{\nu \Lambda\, B\, h(z)}_{\text{diffusion, exact}} \Big) \Big]$$

The per-iteration cost

Decode at m nodes only ($m \times R$), one upwind stencil on m values, an $M \times m$ matvec, and the exact linear part. **Independent of the grid size n.** Measured: latent solve flat from $N = 128$ to 4096.

Choosing the nodes: baseline = NNLS

Given positions X, weights are fit by non-negative least squares so the sampled term reproduces the exact full-grid term on training states. Convex, no gradients, always available as a fallback. Positions themselves are the free choice — stage 2 learns them.

Stage 2 — learning where to sample (decoder frozen)

Only the node positions X are trainable. The loss is a **mismatch against the exact full-grid term**, in value *and* in the z-gradient the solver follows:

$$L_{\text{samp}} = \text{mean}_s \frac{\left\| W_s \left(\Phi_X^\top (w \odot N_X(u_s)) - \Phi^\top N(u_s) \right) \right\|^2}{\| W_s \, \Phi^\top N(u_s) \|^2}, \qquad L_{\text{jac}} = \text{same expression for } \partial_z(\cdot)$$

$$L_2 = \frac{L_{\text{samp}}}{L_{\text{samp}}^0} + \frac{L_{\text{jac}}}{L_{\text{jac}}^0} + w_{\text{sep}} \sum_{i<j} \text{relu}\left(d_{\min} - |x_i - x_j| \right)^2$$

Optimisation

- Adam on X only, LR 3×10^{-3}, 2 000 steps.
- A sigmoid box keeps every node inside the domain.
- The last term keeps nodes apart (minimum separation).
- Each term $\div$ its starting value: both begin at 1, equal weight.

Weights are never gradient-trained

Every 500 steps the weights $w \geq 0$ are re-solved by NNLS on the same loss rows (variable projection). The convex NNLS rule is therefore always the fallback: learning can only move points, never damage the decoder.

Stage 2 — the design choices, and why

Teacher frozen

The full-grid advection term and the decoder cannot move, so the loss cannot be gamed by "fooling the points" — only better placement lowers it. (An earlier co-training variant that let the decoder move was negative.)

Match the gradient, not just the value

The solver steps along $\partial r/\partial z$. A rule that matches residual values but not their z-sensitivity can send the solver the wrong way — Poisson's NNLS rule had gradient cosines below zero.

Held-out states

A disjoint set of training states sits in no gradient step and no NNLS row; it monitors whether the placement generalises rather than memorises.

What the certification is

Rollouts on 8 fresh test trajectories the decoder never saw, graded against the full-order truth, compared to the same-budget NNLS rule and to the full-grid oracle.

1D Burgers results ($N = 128 \ldots 4096$, 8 fresh trajectories, 50 implicit steps)

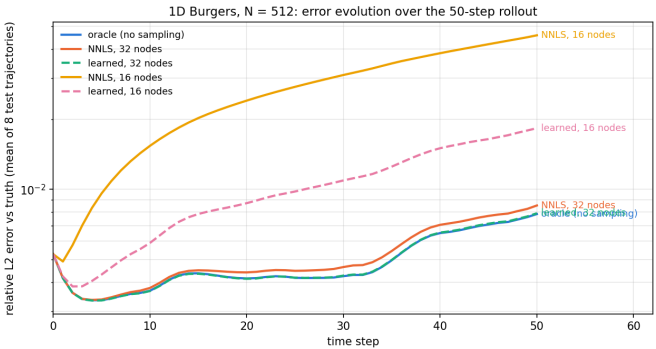
N	decoder floor	oracle	NNLS 32	learned 32 (vs NNLS)	NNLS 16	learned 16 (vs NNLS)
128	3.71e-3	5.01e-3	5.48e-3	5.02e-3 (−8.4%)	2.10e-2	1.01e-2 (−52%)
256	3.73e-3	6.17e-3	6.24e-3	6.20e-3 (−0.6%)	2.56e-2	1.07e-2 (−58%)
512	3.80e-3	4.88e-3	5.19e-3	4.89e-3 (−5.9%)	2.68e-2	1.03e-2 (−62%)
1024	3.79e-3	5.54e-3	5.62e-3	5.45e-3 (−2.9%)	2.22e-2	8.61e-3 (−61%)
2048	3.83e-3	5.08e-3	5.53e-3	5.15e-3 (−7.0%)	2.25e-2	8.44e-3 (−63%)
4096	3.83e-3	4.49e-3	4.89e-3	4.53e-3 (−7.3%)	2.29e-2	9.22e-3 (−60%)

Oracle = advection summed on the full grid (no sampling): the best any node set can do with this decoder. Flat across 32× in resolution.

32 nodes: sampling costs 2–10 % over the oracle; learned nodes close 84–97 % of that gap — they land on the oracle.

16 nodes (starved): NNLS is 4–5× the oracle error; learned placement cuts it by 52–63 %, six for six. It improves a budget; it does not replace a bigger one.

Error over the rollout ($N = 512$)



- All arms share the same initial-condition fit (step 0).
- Oracle and learned-32 are indistinguishable step for step.
- The starved NNLS rule (16 nodes) drifts from step 1 onward (4.6e-2 by step 50); learned nodes at the same budget hold it to 1.8e-2.

ROM online cost per 50-step trajectory (A100), independent of resolution: 49–62 ms; 22–24 ms after inner-loop optimisation with errors unchanged to 10^{-9}.

Extending the linear trick one degree up

Recall why linear was free

$\Phi^\top(G\,h) = (\Phi^\top G)\,h = B\,h$ — a weighted sum of a mix is the mix of the weighted sums.

A **quadratic** term is a weighted sum of a product of two mixes. Expand both mixes and you get every *pair* of shapes — but there are only $R^2 = 1024$ pairs, and none of them depend on z:

$$\Phi^\top\left(u \odot D^- u\right) = \sum_{j,k} h_j\, h_k \underbrace{\sum_x \Phi_{xi}\, G_{xj}\, (D^- G)_{xk}}_{T_{ijk}} \;=\; h^\top T\, h, \qquad T \in \mathbb{R}^{M\times R\times R} = 32\times 32\times 32.$$

Same shopping analogy, one level up

The cart total now depends on *pairs* of products ("buy j and k together"). There are 32×32 pairs; price every pair once, and any cart is a 32×32 lookup times the amounts. Still no need to walk the aisles.

What runs online

One contraction $h^\top T h$ (∼30k flops) and its Jacobian $(Qh)\,\partial h/\partial z$, $Q = T + T^{(j\leftrightarrow k)}$ (T is not symmetric: bank vs. differenced bank). No grid, no sample points, no fitted weights, nothing trained. T is built **once**, offline — 304 ms even at $N = 65\,536$.

Standard precomputed quadratic operator of POD–Galerkin / operator inference; new here only in the setting (nonlinear MLP head over a learned bank). Literature name: *exact projection of a polynomial nonlinearity, no hyper-reduction needed*.

The obstacle was the stencil, and positivity removes it

The FOM's advection is **sign-upwind**: it looks left where $u > 0$ and right where $u < 0$. A switch is not a polynomial — no table can hold it. But the switch decomposes exactly:

$$N_{\text{upwind}}(u) \;=\; u\, D^{c} u \;-\; \tfrac{\Delta x}{2}\,|u|\,\Delta_h u.$$

Central difference plus a small artificial viscosity: the only non-polynomial piece is $|u|$.

On non-negative fields, $|u| = u$

Our 1D family is positive Gaussian blobs with zero walls; viscous Burgers keeps such data ≥ 0 (max principle), and so does the FOM. So on the *truth*, the backward-difference table T **is** the full-grid oracle, exactly. Gate T0: on decoded states that are entirely positive, agreement $\leq 8 \times 10^{-15}$.

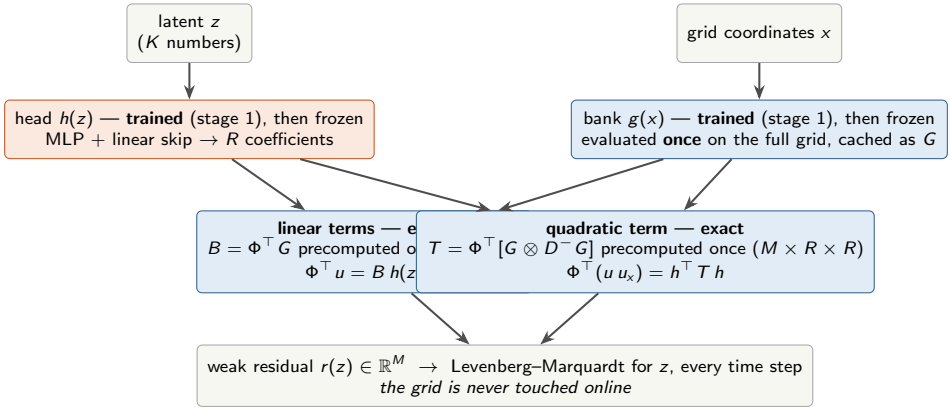
The only deviation: decoded undershoots

The ROM's decoded field dips slightly below zero at ∼7 % of points (min ≈ -0.02). There the FOM would flip the stencil and the table does not. The exact discrepancy is

$$q_T - q_{\text{or}} = -\Delta x\, \Phi^{\top}\big[\mathbb{1}[u \leq 0]\; u\, \Delta_h u\big],$$

first order in the undershoot, times Δx, times the blob's curvature — small because Δx is small. Bounded; measured at 0.4× the bound.

Architecture with the tensor — what changes, what does not



Training: nothing new. Stage 1 is unchanged (same loss L_1, same 40 000 steps, same checkpoints — the results below reuse the *committed* decoders, nothing retrained).

Stage 2 disappears. No node positions to learn, no NNLS weights to fit, no budget m to choose, no held-out monitoring. The orange "nodes" box and the green "weights" box of the earlier diagram are gone; the blue path is the whole residual, as for Poisson.

Results: the tensor lands on the oracle ($N = 128 \ldots 1024$, committed checkpoints)

N	oracle err	**tensor err**	|diff| max/traj	NNLS-32 err	learned-32 err	e2e oracle	**e2e tensor**	e2e NNLS-32
128	5.009e-3	**5.009e-3**	1.2e-6	5.480e-3	5.021e-3	29.0 ms	**24.4 ms**	25.0 ms
256	6.165e-3	**6.165e-3**	6.4e-7	6.236e-3	6.197e-3	35.0 ms	**29.2 ms**	29.5 ms
512	4.877e-3	**4.877e-3**	2.5e-7	5.194e-3	4.886e-3	30.7 ms	**27.5 ms**	26.1 ms
1024	5.537e-3	**5.537e-3**	2.3e-7	5.615e-3	5.451e-3	27.4 ms	**21.8 ms**	22.0 ms

Accuracy. Equal to the full-grid oracle per trajectory to $\leq 1.2 \times 10^{-6}$ (direct field difference $\leq 9 \times 10^{-6}$); the solver takes the same number of Jacobian evaluations on 31 of 32 trajectories. Zero sample points.

Versus sampling. The tensor inherits the oracle's small margin over NNLS-32 — *not* distinguishable from zero at 8 trajectories (sign test $p \geq 0.07$). Equal to learned-32. The gain is exactness and simplicity, not accuracy at this budget.

Cost. Within $\sim$7 % of the sampled rule end-to-end; 10–20 % under the oracle. No 1D speedup — everything here is launch-bound (see next slide). f64 required: in f32 the table's round-off would exceed the undershoot term.

Results: the latent solve does not grow with N

One A100, one process, N alternating, reps outermost — latent solve, ms per 50-step trajectory

N	oracle (full grid)	NNLS-32	**tensor**
128	21.4	16.5	**17.1**
256	26.2	19.9	**20.2**
512	21.7	14.8	**15.7**
1024	21.3	15.5	**16.0**
2048	20.4	15.9	**14.4**
4096	20.8	13.6	**14.3**
exponent	−0.04	−0.07	**−0.08**

Beyond the launch floor (separate jobs, same GPU model; decoders trained in-job at the same 3.76×10^{-3} floor)

N	oracle	NNLS-32	**tensor**	tensor = oracle to
16 384	21.4	13.9	**14.2**	5.8×10^{-10}
65 536	32.9	13.5	**13.8**	1.7×10^{-10}

Why the oracle looks flat at first

Each solver iteration is ∼15 tiny GPU operations, each with a 5–10 μs launch overhead. Decoding 4096 points takes $< 1\,\mu$s of actual work — invisible. Below $\sim 10^4$ points wall time counts *operations*, not bytes: the "launch floor".

Why the tensor stays flat forever

Its per-iteration work is a 32^3 table times a 32-vector. N is not in its shape. Only the offline build sees the grid.

Honest scope

"Constant" = the *latent solve*. The IC projection and the final decode still touch the grid once per trajectory (6–9 ms + $<$ 1 ms here, flat only because launch-bound). The 16k→65k numbers are a cross-job ratio, not a fitted exponent.

What five adversarial reviews said (3 Claude, 2 Codex) — and where the method stops

Verdicts: "legitimate with corrections" ×5, none blocking

- No leakage: T from bank + modes only; oracle is the committed baseline path; truth regenerated from seed, test set seed-disjoint.
- One reviewer reproduced every printed digit on CPU.
- Corrections applied: "second order" → first order with a bound; "identical decisions" → 31/32; "beats NNLS by 1–9 %" retracted; arm order was sequential, not interleaved; cross-job exponents → ratios; f64 required.
- Framing: standard quadratic Galerkin/OpInf operator; novelty is the MLP-head-over-learned-bank setting.

Where it stops

- **Sign-changing data:** the $|u|$ term is not tabulated. Routes: a polynomial FOM (centered / skew / Lax–Friedrichs), tabulate the central part and sample only the small $|u|$ term, or lift $v = |u|$.
- **Non-polynomial physics** (limiters, WENO, e^u): no finite table; sampling stays the right tool, or lift.
- **2D as it stands:** the 2D Burgers bank is $R = 512$, so T would be $128 \times 512^2 \approx 270$ MB — needs compression (project h onto the $\sim K$ directions it spans) before a 2D port.
- **Statistics:** single seed, 8 trajectories; multi-seed with paired intervals is still open.

Linear vs nonlinear, side by side

	Linear (Poisson)	**Quadratic, positive data (Burgers, tensor)**	**Non-polynomial / switched (sam-pled)**
Residual in $h(z)$	$\Lambda B\, h - f_M$, linear	B-terms + $h^\top T h$, quadratic	linear terms via B; the rest evaluated pointwise
Grid folded away?	entirely (B once)	entirely (B and T once)	only the linear terms
Online per iteration	head + 64×32 matvec	head + 32^3 contraction	head + decode at m nodes + stenci + matvec
Sample points	**0**	**0**	m (32 / 16 / 128)
Residual approximation	none (10^{-13})	none on $u \geq 0$; $O(\Delta x\, \delta)$ on under-shoots	quadrature error (NNLS + learned nodes)
After stage 1	nothing	nothing (build T once)	NNLS weights + stage-2 node posi-tions
Accuracy limit	decoder floor	decoder floor (= oracle)	decoder floor + sampling gap
Cost vs grid size	flat	flat (solve $128 \to 65\,536$)	flat

One sentence

The decoder is linear in its last stage, so every *polynomial* piece of the physics can be pre-projected onto the test modes once — linear into a matrix, quadratic into a 3-index table; only a genuinely non-polynomial piece (a switch, a limiter, e^u) must be sampled, and there we learn where to sample.

Glossary (1/2)

Latent z The K numbers (8 or 16) that stand in for a whole field.

Bank g, head h The two halves of the decoder: shapes in space (g; table G once evaluated) and mixing amounts from z (h).

Test modes Φ, M The M sine functions the residual is projected onto (weak form); eigenvectors of the discrete Laplacian with eigenvalues Λ.

$B = \Phi^\top G$ The precomputed $M \times R$ table that turns any linear grid sum into a small matvec.

Quadrature Estimating a grid-wide sum from m sample points with weights. **Quadrature-free** = the sum is computed exactly without points.

Full grid / oracle The residual (Poisson) or the advection term (Burgers) summed over every grid point — the reference the sampled rules try to match; costs $O(N)$ per evaluation.

Tensor T / Q The precomputed $M \times R \times R$ table that gives $\Phi^\top(u\, u_x)$ as $h^\top T h$; Q its symmetrised form for the Jacobian. Built once offline; makes the quadratic term exact with zero sample points.

Undershoot Decoded field values slightly below zero where the truth is ≥ 0; the only place the tensor and the sign-upwind oracle differ.

Launch floor The regime where each GPU operation finishes faster than the next can be launched, so wall time counts operations, not data; every arm sits on it below $\sim 10^4$ points.

Glossary (2/2)

NNLS	Non-negative least squares: fits the node weights so the sampled term reproduces the full-grid term on training states; convex, no gradients.
Learned nodes	Node positions optimised by gradient (stage 2) with the decoder and teacher frozen; weights still by NNLS.
Tight / starved / generous budget	$m = 32$ ($= M$), 16, 128 nodes.
Decoder floor	The error of the best possible latent for each truth snapshot; nothing downstream can beat it.
Rollout error	Relative L_2 error against the full-order truth over the 50 implicit time steps, mean of 8 fresh test trajectories.
Gate	An in-job numerical identity check (e.g. gate Q: quadrature-free residual = full-grid residual to machine precision at every solution; gate T0: tensor = oracle on all-positive states).
LM attempt	One iteration of the Levenberg–Marquardt solver (residual + Jacobian + trial step); $\sim$200 per 50-step trajectory.